

RESEARCH ARTICLE

Hardware Software Co-Design for Multi-Threaded Computation on RISC-V-Based Multicore System

BINH KIEU-DO-NGUYEN¹, (Graduate Student Member, IEEE),
KHAI-DUY NGUYEN¹, (Graduate Student Member, IEEE),
NGUYEN THE BINH¹, (Graduate Student Member, IEEE),
KHAI-MINH MA^{2,4}, (Graduate Student Member, IEEE),
TRI-DUC TA^{3,4}, (Graduate Student Member, IEEE),
DUC-HUNG LE^{2,4}, (Member, IEEE),
CONG-KHA PHAM¹, (Senior Member, IEEE),
AND TRONG-THUC HOANG¹, (Member, IEEE)

¹Department of Computer and Network Engineering, The University of Electro-Communications (UEC), Tokyo 182-8585, Japan

²Department of Electronics, Faculty of Electronics and Telecommunications, University of Science, Ho Chi Minh City 72711, Vietnam

³Department of Integrated Circuit and Hardware Design, Faculty of Computer Engineering, University of Information Technology, Ho Chi Minh City 71300, Vietnam

⁴Vietnam National University Ho Chi Minh City, Ho Chi Minh City 71300, Vietnam

Corresponding author: Duc-Hung Le (ldhung@hcmus.edu.vn)

This work is funded by Vietnam National University Ho Chi Minh City (VNU-HCM) under grant number DN2024-18-02.

ABSTRACT The open-source and customizable features of the RISC-V Instruction Set Architecture (ISA) have facilitated its rapid adoption since its publication in 2011. The availability of numerous free core designs leads to the pervasiveness of RISC-V-based devices on diverse applications spanning the Internet of Things (IoT), embedded systems, artificial intelligence (AI), and virtual/augmented reality (VR/AR). The increasing prevalence of RISC-V cores has consequently caused a demand for high-performance and resource-efficient multicore systems. However, while numerous proposals exist for constructing multicore systems on conventional architectures, realizing an efficient multicore system that effectively leverages the features of RISC-V remains a challenge. This paper introduces a novel hardware/software co-design methodology to address these bottlenecks while minimizing resource utilization. Experimental results demonstrate the efficiency of our approach, exhibiting significant performance gains over single-threaded implementations and even surpassing traditional multi-threaded approaches.

INDEX TERMS RISC-V, multithread, multicore, task scheduling, hardware-software co-design.

I. INTRODUCTION

RISC-V is expected to be used in the specialized processor cores that will soon emerge in wearables [1], home appliances [2], and robots [3]. The need for RISC-V-based multicore platforms is becoming increasingly urgent. Currently, there are various types of RISC-V cores [4], [5] or RISC-V development frameworks [6], [7] that have been released. Using them makes it possible to configure

them as customized multicores for various applications, such as Artificial Intelligent [8], Machine Learning [9], edge computing [10], Radio Frequency (RF) applications [11]. However, in terms of practicality, creating a multicore platform using the open RISC-V cores poses enormous challenges. This is mainly due to the coherence [12] and the bottleneck when using shared components, buses, and memory systems [13], [14].

One potential solution to address this challenge is the implementation of thread management systems. Thread management effectively promotes parallelism due to reduced

The associate editor coordinating the review of this manuscript and approving it for publication was Liang-Bi Chen¹.

inter-thread dependencies, facilitating optimal collaboration between software and hardware for maximized performance. Thread management can be implemented at either the operating system level [15] or the software level [16]. However, this method necessitates additional software development effort without specialized hardware support, and it requires one core dedicated to monitoring the main thread as [16].

In contrast, the authors in [17] introduced a dual-threaded single-core processor featuring two distinct register files and context switch mechanisms. A context switch is triggered on stall detection to fetch and execute another thread. Nonetheless, this approach incurs time and energy overheads by many context switches, and only one thread can be executed at any given time. Another strategy involves employing a complex bus system to manage data flow when multiple cores use shared resources [8], [18], [19]. While this approach demonstrates notable performance gains, the complexity and power consumption of the bus system increases rapidly with the number of cores and connected shared devices. Consequently, such buses are only utilized for managing shared memory systems, which often represent the primary bottleneck in high-performance systems.

The most efficient method targets processing in or near memory. Two primary approaches exist within this paradigm. The first targets proposing efficient memory designs [20], [21], [22], while the second focuses on computing data stored in the cache. Cache offers high bandwidth with reduced access energy [22]. The authors in [23] concentrate on the Last Level Cache (LLC), which corresponds to the Level 2 (L2) Cache in their proposed system. Processing at the LLC enables handling larger datasets but necessitates additional accelerators and does not fully exploit the capabilities of RISC-V cores.

Conversely, the authors in [14] leverage the L1 cache, a hybrid cache that bypasses the bottleneck associated with multicore access. This method shows remarkable speedup and low power consumption despite requiring a complex hardware implementation and being primarily suitable for many-core architectures.

We've developed a new approach to simplify multicore development on RISC-V platforms. We focus on minimizing bottlenecks in shared resource access and optimizing thread management through a combined hardware/software solution. While developers have some control over data caching, we provide simple APIs to simplify integration. Additionally, we analyze and suggest an approach for various parallel algorithms that can benefit from our hardware. Our solution is adaptable to any RISC-V design and core configuration, ensuring real-world applicability. We validated its effectiveness by implementing a RISC-V multicore prototype on an FPGA and testing several applications. The results demonstrate a remarkable 400% performance improvement over platforms without our method. In summary, our key contributions include:

- A minimal configuration that enables multi-threads on a multicore RISC-V-based system. The multicore system

can work independently or become a part of many-core system [24], [25].

- A design of hybrid cache as additional L1 Data Cache to reduce the bottleneck when accessing shared cache (L2) and bus, a serious issue in multicore systems.
- An approach to take advantage of the proposed multi-threads-enabled RISC-V system with a software/hardware co-design approach.

The remainder of this paper is organized into five sections. Section I introduces our works with a literature review. Section II presents the hardware implementation of a multicore RISC-V-based system with a minimal configuration that supports multi-threads. Section III introduces the main idea of implementing the intensive workload algorithms on the proposed multicore system. Next, section IV is to develop a prototyped RISC-V multicore platform and applications as testbenches and provide experimental results. Finally, Section V concludes the paper.

II. HARDWARE IMPLEMENTATION

A. PROPOSED MULTICORE SYSTEM

Figure 1 illustrates our proposed multicore RISC-V system design. The design aims for parallel processing efficiency and adaptability across various applications with optimized configuration.

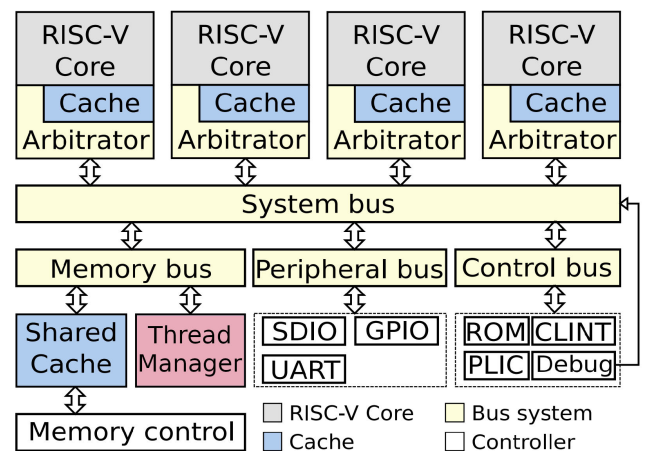


FIGURE 1. System architecture with four RISC-V cores.

The system consists of four asymmetric RISC-V cores, each implementing a Reduced Instruction Set Computing V (RISC-V) Instruction Set Architecture (ISA). Asymmetric configurations leverage the modularity of RISC-V ISA to achieve the highest performance with less resource and energy consumption [26]. Each core integrates a hybrid Level 1 (L1) cache, a small but high-speed memory, which is pivotal in optimizing core performance, thereby reducing delay and enhancing overall data access.

Arbitrators, in addition to the L1 cache, act as traffic controllers for the shared system bus. These arbitrators ensure non-conflictual access to the bus, preventing any single core from dominating the shared resources, which may result in

congestion and reduce performance. The system bus itself is a network with three distinct hierarchies:

- The memory bus handles the exchange of data and instructions among the computational cores and the main memory. It also includes the shared L2 cache, a buffer for data fetched from external memory. The memory bus interfaces with a Thread Queue, which stores the thread information produced by the main core. The Thread Queue enables atomic access to the queue to guarantee the coherence of read/write operations.
- To accommodate the system's various input/output requirements, the peripheral bus interfaces with various peripheral devices. The pins encompass SDIO (Secure Digital Input/Output) often employed for SD cards, UART (Universal Asynchronous Receiver/Transmitter) enabling serial communication, and GPIO (General-Purpose Input/Output) pins for versatile connection with diverse sensors, actuators, and external components.
- The control bus carries essential control systems such as Read Only Memory (ROM) that stores information for the boot process, Core Local Interrupt (CLINT) that manages the software and timer interrupt, Platform-Level Interrupt Controller (PLIC) takes responsibility for peripherals' interrupts and Debug module to develop and debug the deployed software.

In addition to the hybrid L1 caches, there is a shared L2 cache that all cores may access. This cache functions as a storage for data that several cores may request. By sharing the cache, the strain on the main memory is significantly reduced, therefore enhancing the system's overall performance. A thread queue controls and distributes the generated threads, the fundamental execution units in the system, thus guaranteeing effective task execution and resource economy. This queueing system guarantees fairness in task allocation and keeps any thread from starving others of processing capability.

As the backbone, the bus system supervises the entire bus system and ensures coherence and optimal performance in the architecture. The system orchestrates the operations of the arbiters, memory controller, and other components, guaranteeing data transmission and avoiding conflicts or shared resource accesses.

The RISC-V system integrates the necessary peripherals for system operation and troubleshooting. The components of the system consist of ROM (Read-Only Memory) for the storage of firmware and boot code, CLINT (Core-Local Interrupt) for the management of timers and software interrupts, PLIC (Platform-Level Interrupt Controller) for the control of external interrupts, and a specialized debug interface for troubleshooting and system diagnostics. The RISC-V architecture's simplicity, modularity, and flexibility offer diversity for the multicore system. From resource-limited embedded systems to high-performance computing clusters, its adaptability and simplicity let it easily customize and fit for many uses.

B. THE RISC-V CORE ARCHITECTURE

Figure 2 depicts the architecture of a RISC-V core in our proposed multicore system, a compact microprocessor designed for effectiveness and flexibility. The pipelined architecture is applied to efficiently coordinate the execution of instructions by dividing the process into stages: Fetch, Decode, Execute, Memory Access, and Write Back. Several instructions can be executed simultaneously through parallel processing, significantly improving performance.

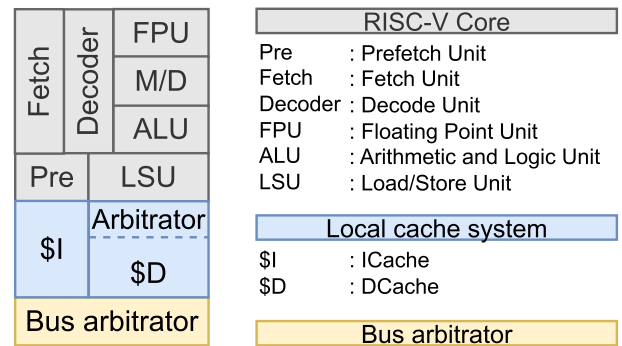


FIGURE 2. Configuration for a single core.

The Fetch unit fetches instructions from either the instruction cache, a high-speed memory for frequently used instructions, or a prefetch unit, which predicts and fetches the following instructions. When instructions are fetched, the Decode unit decodes them by extracting the operation code (Opcode) and identifying the operands.

After decryption, the instructions are dispatched to the unique execution units. This unit performs all algebraic operations (addition and subtraction) and logical operations (including AND, OR, and XOR). In the core, the Load/Store Unit (LSU) controls the data transaction between the registers and memory. The Multiplier Unit (MUL) controls polynomial operations, demanding enormous computational resources. A floating point unit (FPU) can be optionally integrated to execute operations on real numbers for applications needing accurate computations. FPU is not required. However, it has advantages for many uses and is essential for compiling important libraries.

The hybrid DCache is integrated as an addressable memory of the RISC-V core. It acts as a regular cache or local memory for the RISC-V core that stores the temporary variable produced during the thread's performance. The hybrid cache helps to reduce the bottleneck when multiple cores try to store hybrid data, which are usually temporary variables while performing their thread.

C. HYBRID DATA CACHE

Figure 3 illustrates our proposed hybrid cache architecture, highlighting the interaction among the RISC-V core, data cache, and main memory through the shared bus. DCache is a small but fast on-chip memory that stores copies of

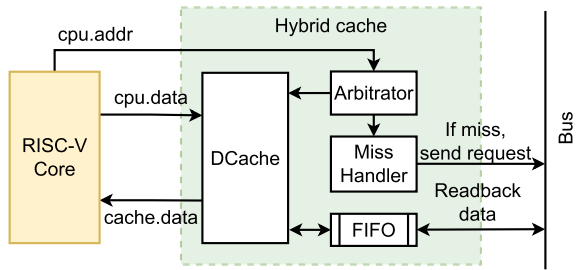


FIGURE 3. Hybrid cache architecture.

frequently used data from the main memory. In addition, our proposed hybrid cache architecture is addressable. This means that the running software on the RISC-V core can directly access DCache through its assigned address. In this way, the RISC-V core can use the local hybrid cache as its main memory to ignore the delay of cache replacement and the congestion when using the shared bus. The hybrid cache is used as a standard cache in the regular cases.

When the RISC-V core requests data, it sends the memory address (*cpu.addr*). The Arbitrator first decodes the memory address. If the requested address is the assigned address range for the hybrid local cache, the requested address is forwarded to access data in the DCache directly. In this case, the D-cache is used as the main memory. Otherwise, the requested address is forwarded to Miss Handler. The Miss Handler checks if the DCache holds the requested data. If found (a cache hit), the data (*cache.data*) is returned to the CPU (*cpu.data*). If the data is not in the cache (a cache miss), the Miss Handler requests the missing data to the main memory through the system bus. A FIFO (First-In, First-Out) buffer is deployed to manage burst transactions.

The optimal size of the hybrid DCache is recommended from 8 to 32 kB, which is the typical L1 Cache size of modern CPUs. More cache size can increase performance, but it also increases the size and cost of the final product.

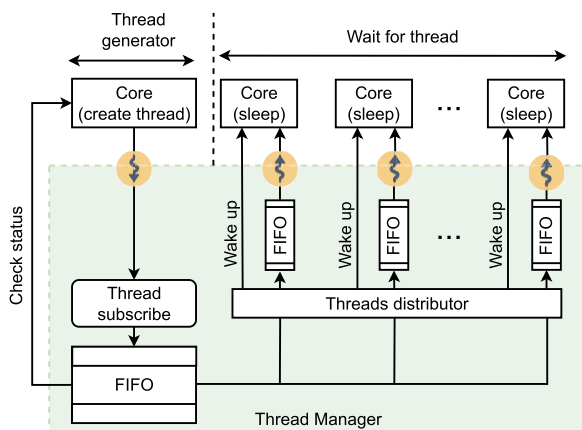


FIGURE 4. Thread manager architecture.

D. THREAD MANAGER

Figure 4 illustrates the proposed thread management system for optimal performance and resource utilization in a multicore processor. In our proposed system, there is a core responsible for creating new threads called the main core. The main core creates threads by subscribing the thread's information to the income thread's FIFO, such as the address of the subscribed function, the addresses of the passing parameters, and the identification (ID) of the core assigned to perform this thread. A specific ID indicates that the corresponding core will perform the subscribed thread. A broadcast ID indicates that any core could perform the subscribed thread.

The core of the Thread Manager is the Threads distributor. The Threads distributor is the central controller, monitoring the main FIFO of income and outcome threads. It can detect if there is any subscribed thread in the income thread FIFO and then fetch the first available entity. Then, it distributes the thread to a specific core if this thread is assigned for a particular ID. Otherwise, it distributes the thread information to a sleeping core of the core with the lowest number of waiting threads. When assigning a thread for an empty outcome FIFO, the Threads distributor also wakes the corresponding core from sleep. This ensures that the workload of each core is balanced. In this way, the Thread Manager offers a friendly way to manage multiple threads and allows flexibility when the software can actively assign threads for a specific core.

Each core operates independently, retrieving threads from its dedicated FIFO queue. On completion of a thread's execution, the core will check the stage of the corresponding FIFO. If the FIFO is empty, the core enters a low-power "sleep" state, effectively conserving energy and minimizing resource consumption. This "sleep" mode is crucial for optimizing energy efficiency, especially in systems with many cores.

The size of the income and outcome thread's FIFO depends on the available resources of the deployed system. When the outcome FIFO is complete, the Threads distributor stops fetching from income thread FIFO. On the other side, before the main core subscribes thread, it checks the income FIFO state first. If the income FIFO is full, the main core waits until the income FIFO has space, or it can perform its waiting threads before generating the thread again. The "create thread" method is implemented in the firmware, including checking the income FIFO status and subscribing thread's information.

III. ADAPTING SOFTWARE

A. THREAD MANAGEMENT

As the most practical solution to developing multicore with RISC-V, this paper proposes a software-based scheme that can take advantage of our multicore system without developing a new compiler. Figure 5 shows the motivation

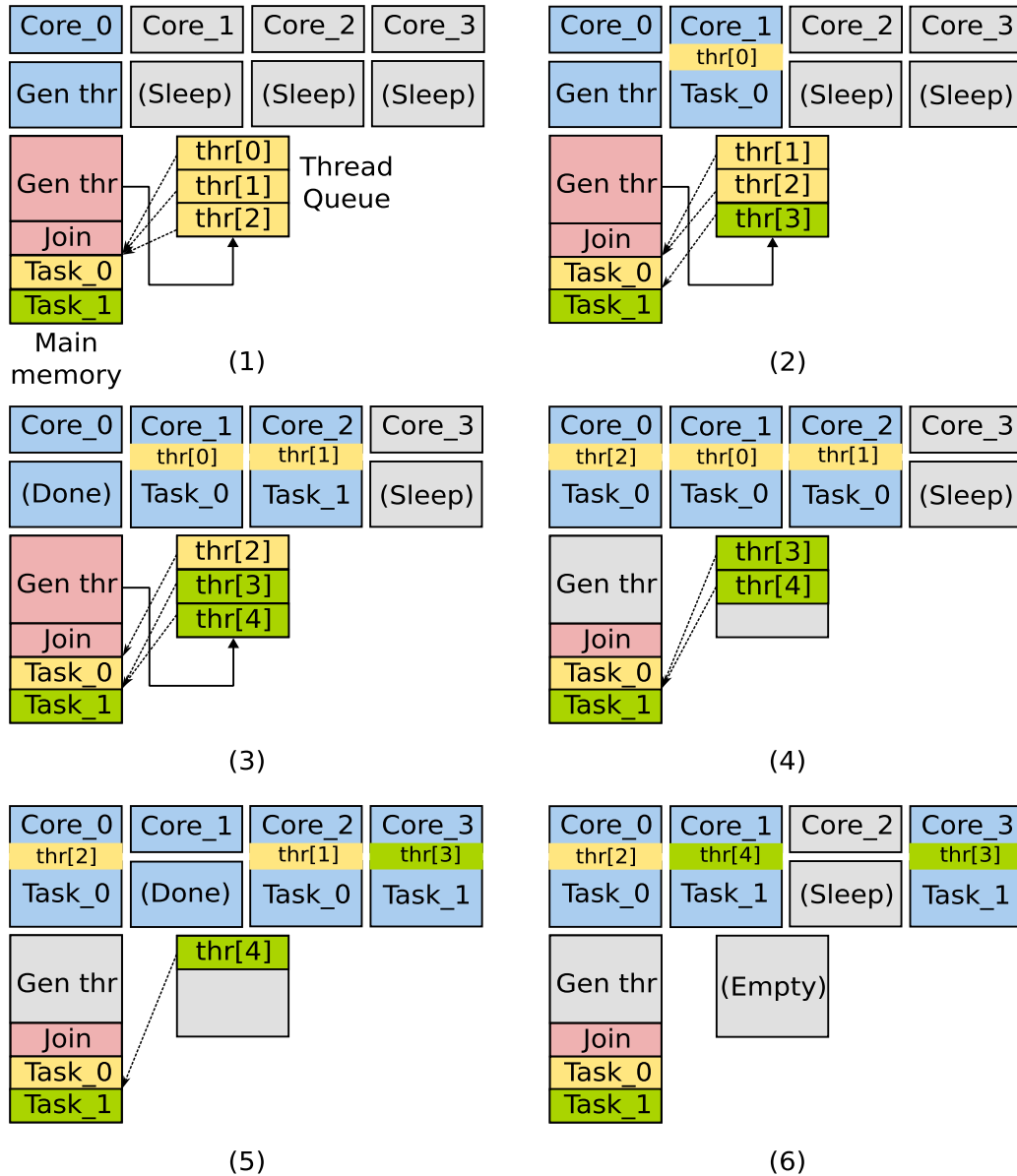


FIGURE 5. Thread management by hardware/software co-design.

and main idea of the proposed scheme. Figure 5 also illustrates a detailed hardware/software co-design approach for dynamically managing threads across a multicore system, likely a RISC-V architecture. This system leverages four cores (Core_0 to Core_3) and a shared Thread Queue to maximize parallelism and efficiency. The proposed scheme is based on our suggested RISC-V multicore system and RISC-V architecture. In the general case, it could be applied for multiple cores, not only four, as in the following example. Our thread management scheme follows six steps:

- 1) Initialization: The primary thread executes a Generate Threads (Gen thr) to produce new threads, which are then added to the Thread FIFO. The sub-cores (the cores that do not perform primary thread) are first inactive and waiting to be assigned a task. The thread's

specification contains an index of the core that has been granted permission to execute this thread and the starting address of the associated function. The core index may represent either a specific core or all cores, all of which are equally accessible.

- 2) First fetch: The subscription of the thread by the primary thread wakes the sub-cores from sleep. The sub-cores request thread information from the Thread Queue. Only one sub-core has the right to access the Thread Queue at the moment. After getting the thread information, this core releases control of the Thread Queue and performs the fetched thread.
- 3) Parallel Execution and Thread Generation: As Core_0 continues generating new threads (thr[3], thr[4]). Simultaneously, Core_1 continues its work on thr[0],

while Core_2 wakes up and fetches the previously generated thr[1]. Core_3 remains in sleep mode, waiting for its turn. Only the primary core has written permission, while the other cores can read only.

- 4) Joining: Core_0, after finishing threads' generation, takes thr[2] on the Thread Queue. This is a crucial point of our proposed thread control flow; it allows the primary core (Core_0) to turn into a sub-core to maximize the parallelism of the multicore system.
- 5) Task Completion and Queue Depletion: Core_1 completes thr[0] and immediately fetches thr[4], ensuring maximum utilization. Core_3 finally joins the array, taking on thr[3].
- 6) Finalization and Resource Release: Core_1 gets the thr[4] and empties the Thread Queue. After finishing Task_0, Core_2 turns into a sleep state when detecting the empty state of the thread FIFO. The other cores also sleep after finishing their task and detecting the emptied Thread Queue.

In this hardware/software co-design, the effectiveness of the suggested scheme of thread generation, scheduling, execution, and synchronization is highlighted. The system showcases its ability to adjust to different workloads or numbers of cores efficiently, maximizing the use of all available cores and improving parallel processing capabilities and overall system performance. Executing the Thread FIFO and Join operations strategically guarantees effective load balancing and accurate task ordering, which is essential for intricate, multi-threaded systems.

Figure 6 summarizes the dynamic thread management system across a four-core processor with the supported Thread Manager. Initially, the mater core (Core_0) generates the threads, then is assigned to different cores for parallel execution. The other cores are waked up and fetch the available thread until the thread FIFO is empty.

In a multi-thread environment, threads are created at the start and when other cores operate. This method makes the best use of resources since cores remain occupied as long as threads are available. When nothing is to be done, sleep states for idle cores help save energy.

Figure 7 illustrates the execution flow when a core performs a thread. When a core is wakened up by the Threads distributor (see Figure 4), it first checks its corresponding thread's FIFO. If the FIFO is empty, the working core returns to sleep mode. Otherwise, it saves its current state, which is the value of the register file, then fetches the thread's information, including the address of the function and the address of the passing parameters. Next, the core moves the stack pointer to the assigned address of the hybrid cache. In this way, every local variable will be allocated in the hybrid cache. Finally, it calls the thread function through the passing address. All the steps above are implemented in firmware (see Figure 7). The programmer implements the next steps. After the function of the executing thread is called, the software first copies the passed parameters to the local variables. Then, the implemented program is performed on

them. Finally, the results are written back to the main memory through the passing addresses of the corresponding outputs. The program flow returns to the firmware and restores the core's state. Then, it checks the status of its corresponding FIFO before fetching the new thread or returning to sleep.

The following subsections suggest some schemes to maximize the usage of hybrid cache to mitigate the bottleneck encountered when accessing the shared cache and shared bus, which are the primary challenges of current multicore systems. The examples illustrate the approaches to enhance parallelism: Divide and Conquer, Break the problem segments, and duplicate the computation kernel.

B. MATRIX MULTIPLICATION - DIVIDE AND CONQUER STRATEGY

Algorithm 1 illustrates Divide and Conquer strategy. It revolutionizes matrix multiplication by adopting a Divide and Conquer strategy to reduce space requirements. Given two matrices A and B of size $p \times p$, the algorithm first partitions each matrix into the smaller submatrices.

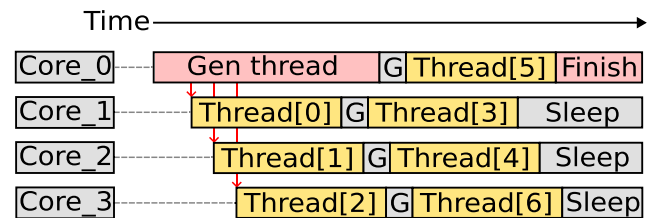


FIGURE 6. Thread subscription timeline.

Instead of performing multiplications on all big matrices, Divide and Conquer strategy reduces the size of inputs to multiple sub-matrices. These products are calculated by multiplying the submatrices of A and B . The matrix multiplication $MatMul$ is defined in Algorithm 2. The power of this strategy relies on the deployed system's parallelism level. These products can be computed simultaneously, potentially leveraging the capabilities of modern parallel computing architectures. Following the calculation of the intermediate products, the algorithm performs parallel addition and subtraction operations to determine the submatrices of the result matrix C . The matrix addition $MatAdd$ is defined in Algorithm 3.

Divide and Conquer matrix multiplication algorithm recursively applies this process to the intermediate products for large matrices, breaking them down until a base size is reached. Finally, the resulting submatrices are combined to form the complete product matrix C . By applying this strategy, programmers split the input data into smaller chunks and store it in the hybrid cache, which can be accessed without delay and does not cause a bottleneck on a shared bus. The data flow when deploying the algorithm on our proposed system is illustrated in Figure 8. Figure 8 demonstrates the data flow of a parallel computing architecture, specifically

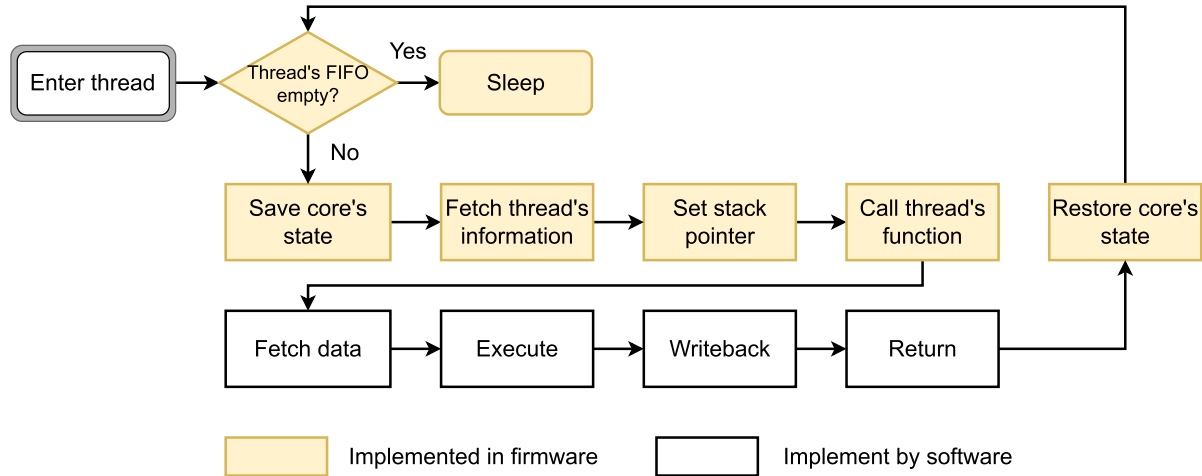


FIGURE 7. Thread execution flow.

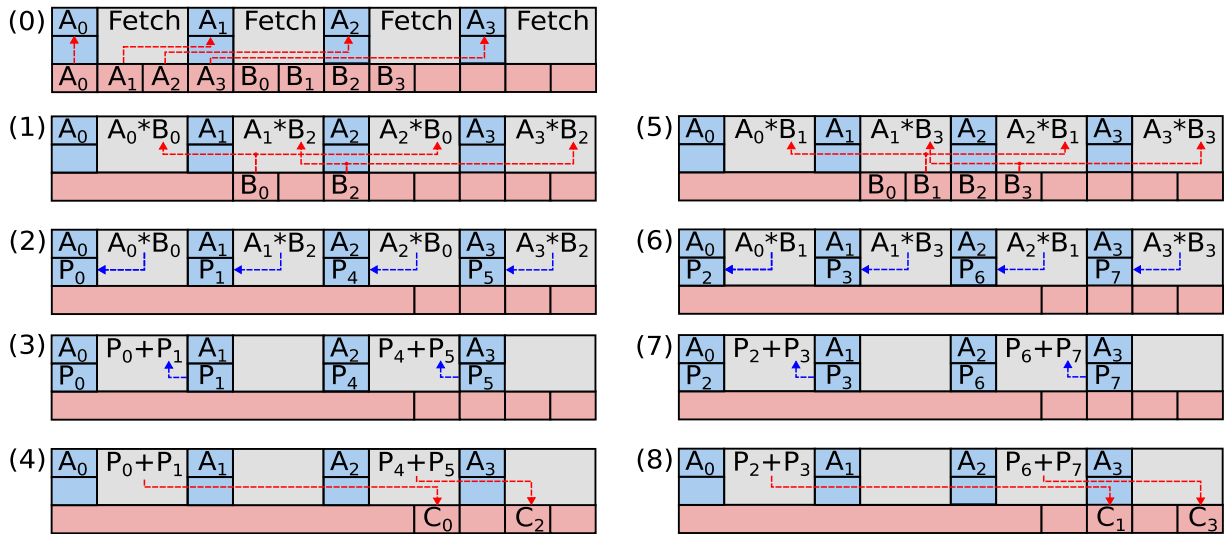


FIGURE 8. Data movement when performing divide and conquer strategy.

when implementing Divide and Conquer strategy for matrix multiplication. The data movement starts with the initialization phase (step 0). Here, the input matrices A and B are divided into submatrices (A_0, A_1, A_2, A_3 and B_0, B_1, B_2, B_3 as described in Algorithm 1), and these submatrices are loaded into the individual RISC-V cores of the multicore system. The submatrices of A are fetched into a hybrid cache, while the submatrices of B are stored in shared memory. In this way, the access of the submatrices of A does not suffer a delay. In addition, it reduces the delay when accessing shared bus and shared L2 cache.

Each core computes an intermediate product based on Divide and Conquer algorithm in the first multiplication phase (steps 1 and 2). These products are derived from specific combinations of the input submatrices, as shown in the diagram. For example, P_0 is calculated as A_0 multiplied

by BS_0 . The resulting intermediate products are then stored in the hybrid cache for reuse in the next steps. Subsequently, the addition phase (steps 3 and 4) takes place. In this phase, the RISC-V cores add the received intermediate products to compute the final submatrices of the resulting matrix C (C_0 and C_2). For instance, C_0 is obtained by adding P_0 and P_1 , C_2 is obtained by adding P_4 and P_5 . These final submatrices are then returned to the main memory through a shared cache. The rest of the result's submatrices are calculated from step 5 to step 8 with the same pattern. However, the initialization phase is skipped because the submatrices of A are still located in hybrid caches. Such a strategy can be scaled up with the number of cores. For example, with eight core configurations, steps 1 to 4 are performed in Core_0 to Core_3, while steps 5 to 8 are performed in Core_4 to Core_7 simultaneously.

Algorithm 1 Divide and Conquer Matrix Multiplication**Input:** matrices A and B of size p .**Output:** C is the result with the appropriate size.

- 1: Partition matrix A of size p into:

$$A = \begin{Bmatrix} a_{0,0} & \cdots & a_{0,p/2-1} & \cdots & a_{0,p-1} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ a_{p/2-1,0} & \cdots & a_{p/2-1,p/2-1} & \cdots & a_{p/2-1,p-1} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ a_{p-1,0} & \cdots & a_{p-1,p/2-1} & \cdots & a_{p-1,p-1} \end{Bmatrix} = \begin{Bmatrix} A_0 & A_1 \\ A_2 & A_3 \end{Bmatrix}$$
- 2: Partition matrix B of size p into:

$$B = \begin{Bmatrix} b_{0,0} & \cdots & b_{0,p/2-1} & \cdots & b_{0,p-1} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ b_{p/2-1,0} & \cdots & b_{p/2-1,p/2-1} & \cdots & b_{p/2-1,p-1} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ b_{p-1,0} & \cdots & b_{p-1,p/2-1} & \cdots & b_{p-1,p-1} \end{Bmatrix} = \begin{Bmatrix} B_0 & B_1 \\ B_2 & B_3 \end{Bmatrix}$$
- 3: Partition matrix C of size p into:

$$C = \begin{Bmatrix} A_0 \times B_0 + A_1 \times B_2 & A_0 \times B_1 + A_1 \times B_3 \\ A_2 \times B_0 + A_3 \times B_2 & A_2 \times B_1 + A_3 \times B_3 \end{Bmatrix} = \begin{Bmatrix} P_0 + P_1 & P_2 + P_3 \\ P_4 + P_5 & P_6 + P_7 \end{Bmatrix}$$
- 4: Parallel multiplication
 - 5: $MatMul(P_0, A_0, B_0)$
 - 6: $MatMul(P_1, A_1, B_2)$
 - 7: $MatMul(P_2, A_0, B_1)$
 - 8: $MatMul(P_3, A_1, B_3)$
 - 9: $MatMul(P_4, A_2, B_0)$
 - 10: $MatMul(P_5, A_3, B_2)$
 - 11: $MatMul(P_6, A_2, B_1)$
 - 12: $MatMul(P_7, A_3, B_3)$
- 13: Parallel addition
 - 14: $MatAdd(C_0, P_0, P_1)$
 - 15: $MatAdd(C_1, P_2, P_3)$
 - 16: $MatAdd(C_2, P_4, P_5)$
 - 17: $MatAdd(C_3, P_6, P_7)$

This dynamic movement of data, supported by a hybrid cache, demonstrates a highly efficient and parallel approach to matrix multiplication. Localized communication contributes to their effectiveness in accelerating computationally intensive tasks by reducing access to shared components such as the cache bus.

C. 2D CONVOLUTION - SPLITTING METHOD

Algorithm 4 outlines the fundamental steps of 2D convolution. The algorithm takes a kernel K , a small matrix representing the filter to be applied, and an input image I as inputs. It then iterates over each pixel in the output image O , calculating its value as a weighted sum of the corresponding

Algorithm 2 Matrix Multiplication $\rightarrow MatMul(C, A, B)$ **Input:** matrices A and B of size p .**Output:** C is the result with the appropriate size.

- 1: **for** i from 0 to $p - 1$ **do**
- 2: **for** j from 0 to $p - 1$ **do**
- 3: $sum \leftarrow 0$
- 4: **for** k from 0 to $p - 1$ **do**
- 5: $sum \leftarrow sum + A_{ik} \times B_{kj}$
- 6: **end for**
- 7: $C_{ij} \leftarrow sum$
- 8: **end for**
- 9: **end for**

Algorithm 3 Matrix Addition $\rightarrow MatAdd(A, B)$ **Input:** matrices A and B of size p .**Output:** C is the result with the appropriate size.

- 1: **for** i from 0 to $p - 1$ **do**
- 2: **for** j from 0 to $p - 1$ **do**
- 3: $C_{ij} \leftarrow A_{ij} + B_{ij}$
- 4: **end for**
- 5: **end for**

pixels in the input image, as determined by the kernel. This step-by-step process ensures that each pixel in the output image is transformed based on the filter's characteristics. 2D convolution is a class of problems that can be easily split into smaller parts [27], [28]. In this scheme, Each core is assigned a distinct filter and is responsible for processing a specific region of the input image. By dividing the task in this manner, the computation can be performed in parallel, leading to significant performance compared to a sequential implementation.

Figure 9 reveals the data flow within the cores during convolution. In step 0, each core fetches its designated filter (K_0 to K_3) into the hybrid cache and loads the corresponding portion of the input image (I_0 to I_3) from shared memory. Each specific fragment of the input image can only be processed by a designated core, but it will not be reused. Therefore, it should not be stored in a hybrid cache.

**FIGURE 9.** Data movement when performing 2D convolution.

Algorithm 4 2D Convolution**Input:** kernel K of size $KSIZE$ input I of size $ISIZE$ **Output:** output O of size $(ISIZE^2 - KSIZE^2 + 1)$.

```

1: for y from 0 to  $ISIZE - KSIZE$  do
2:   for x from 0 to  $ISIZE - KSIZE$  do
3:     sum  $\leftarrow 0$ 
4:     for ky from 0 to  $KSIZE - 1$  do
5:       for kx from 0 to  $KSIZE - 1$  do
6:         in_y  $\leftarrow y + ky$ 
7:         in_x  $\leftarrow x + kx$ 
8:         sum  $\leftarrow sum + I[in\_y \times ISIZE + in\_x] \times K[ky \times KSIZE + kx]$ 
9:       end for
10:    end for
11:     $O[y \times (ISIZE - KSIZE + 1) + x] \leftarrow sum$ 
12:  end for
13: end for

```

The subsequent steps (1 and 2) depict the iterative process of convolution. Each core filters its image portion, generating the corresponding results. These results are then exchanged among cores to ensure the correct computation of neighboring pixels, as convolution involves overlapping regions. This convolution and data exchange process continues until all filters have processed all parts of the input image. Then, the following images can be fetched and processed. The final results, representing the transformed image after applying the various filters, are accumulated to produce the output image (O_0 to O_3).

In conclusion, this multicore approach to 2D convolution exemplifies the power of parallel processing in tackling computationally demanding tasks. This approach leverages the proposed multicore system and RISC-V architecture with a hybrid cache to distribute the workload and optimize data flow. This methodology achieves a significant speedup, making it indispensable for real-time image processing applications where performance is of the essence.

D. CYCLIC REDUNDANCY CHECK 32 (CRC-32) - DUPLICATE KERNEL

Figure 10 illustrates the implementation of the CRC-32 (Cyclic Redundancy Check) algorithm (see Algorithm 5), which is used to ensure data integrity in various applications. CRC-32 operates by generating a unique checksum for a given message, which can then be used to verify the accuracy of the data after transmission or storage. CRC-32 illustrates a class of issues that cannot be performed in parallel. For such problems, each RISC-V can work as a distinct CRC-32 kernel to process different input messages.

At the heart of this implementation is a precomputed lookup table, denoted as T , which stores precalculated CRC-32 values for all possible input bytes. This table is a reference for quick and efficient computation during the

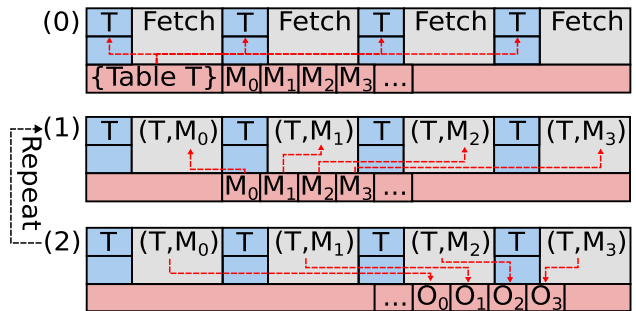
Algorithm 5 CRC-32 ($\oplus$ Is Bitwise XOR; $\sim$ Is Bitwise NOT)**Input:** precomputed table T of size 256input message M of L characters**Output:** output O

```

1: crc  $\leftarrow 0xFFFFFFFF$ 
2: for i from 0 to  $L - 1$  do
3:   crc  $\leftarrow (crc \gg 8) \oplus T[crc \oplus data[i]]$ 
4: end for
5:  $O \leftarrow \sim crc$ 

```

CRC calculation process. Because the table T is used for all CRC-32 processes, it is loaded into a hybrid cache to reduce the shared cache access. At the same time, the input messages are stored in shared memory. The precomputed table is fetched from memory in the initial stage (0). The input characters of each message will be processed sequentially in the subsequent steps at the designated core.

**FIGURE 10.** Data movement when performing CRC-32.

The next stage (1) is the CRC-32 calculation step. Each character from the input message is combined with the current CRC value (either the initial value or the result from the previous iteration). This combined value is used as an index to determine a corresponding value in the pre-computed table 'T'. The fetched value from the table is then XORed (bitwise exclusive OR) with the right-shifted current CRC value. This updated CRC value becomes the input for the next iteration with the next character from the message. This iterative process continues until all characters in the message have been processed.

The final stage (2) involves the output generation. After processing all characters of the input message, the final CRC value is obtained. This value is then inverted (bitwise NOT) to produce the output (O), representing the CRC-32 checksum of the input message.

In the multiple-core system, different cores work as different CRC-32 engines. Then, they process different input messages. The reference data (table T) are frequently accessed, so they are stored in a hybrid cache to reduce the bottleneck when accessing shared bus and memory. This approach takes advantage of the proposed multicore system and RISC-V architecture to significantly improve the throughput.

IV. EXPERIMENTAL RESULTS

A. PROTOTYPE PLATFORM

The experimental system is deployed on the Xilinx VCU118 Evaluation Kit, a Field Programmable Gate Arrays (FPGAs) platform centered around the Virtex UltraScale+ XCVU9P. It offers exceptional performance, integration capabilities, and flexibility, making it suitable for various applications. The kit includes Peripheral MODule (PMOD), which connects JTAG-based debugger, UART interface, and DDR4 as main memory. The Xilinx Vivado Design Suite 2024.1 is used to synthesize, place, route, and generate the bitstream for the experimental system. All experimental programs are compiled by GCC version 12.2.0 for RISC-V with the optimization of level 2 (-O2).

The experimental system is configured as Figure 1 with four Rocket core 64-bit (RV64) with the extensions Integer (I), Multiplication (M), Atomic (A), Floating Point (F and D), and Compression (C). The peripherals include SDIO to fetch programs from SD card, GPIO, and UART. The control modules include the boot ROM, CLINT, PLIC, and a debugger. The debugger communicates with the computer through JTAG and RISC-V GNU Debugger (GDB) support. The main memory is 2.5-GigaByte (GB) of DDR4. The size of the ICache and DCache of each core is 8-KB, and the size of the L2 Cache is 128-KB. The resource usage is explained in Table 1 and the performance is revealed in Tables 2, 3, and 4.

TABLE 1. Resource consumption for a four core system.

	LUTs	Registers	BRAM	DSP
DDR IP	18,330	23,365	25.5	3
Multicore system	55,642	27,596	138	40
Bus system	4,378	1,188	0	0
L2 Cache	4,321	1,881	124	0
Rocket core (*)	11,291	5,568	3	10
Core	2,970	1,499	0	0
MUL/DIV	831	233	0	8
FPU	4,310	1,678	0	2
ICache	1,547	890	1	0
DCache	956	685	1	0
Hybrid cache**	104	118	0	0
Bus	573	465	0	0
Thread queue	143	186	2	0
Other	1,779	2,255	0	0

* Resource for single Rocket core.

** The additional resource of hybrid cache controller.

The total resource of four rocket core need to be multiplied by 4.

B. EXPERIMENTAL RESULTS

The table 1 provides a detailed breakdown of resource utilization within a system-on-chip (SoC) design implemented on an FPGA. The design includes several key components: a DDR IP responsible for interfacing with external DDR memory, a multicore RISC-V-based system, a thread queue for managing the execution of threads, and additional miscellaneous components. The resources under consideration are categorized into Look-Up Tables (LUTs), registers, Block RAM (BRAM), and Digital Signal Processing (DSP) slices. Table 1 also reveals that the DDR IP is

the most resource-intensive element, consuming many LUTs and registers. This highlights the complexity of managing external memory interfaces and their significant impact on the overall FPGA resource usage. Within the multicore system, the Rocket [cite] cores also demand a considerable share of resources. Each core, consisting of the central processing unit (CPU), multiplier/divider (MUL/DIV), floating-point unit (FPU), and various levels of cache memory, contributes significantly to resource utilization. Notably, the MUL/DIV and FPU units within the Rocket cores heavily rely on DSP slices, showcasing their specialized nature and the potential for resource contention if multiple DSP-intensive operations are required. The maximum frequency achieves nearly 178 MHz on Virtex UltraScale+ XCVU9P.

The thread queue and the hybrid cache, which are responsible for distributing threads and reducing bottlenecks when multiple threads access shared resources, utilize significant small resources. This highlights that the proposed design prioritizes efficient task management with a small overhead. This way, our approach is novel for optimizing the multicore RISC-V-based System on Chips for various applications. While the overhead is less than 1% as revealed in 11, many potential strategies, such as integrating the co-processor/accelerators [29], [30], [31] or sharing computation resource [32] to achieve the specific requirements of an application, are available.

Figure 11 provides a breakdown of the resource consumption within a multicore system with four Rocket cores. The analysis focuses on two critical resources: Look-Up Tables (LUTs), the basic building blocks for implementing logic functions in FPGAs, and registers for storing temporary data.

At the multicore system level, each of the four Rocket cores emerges as the dominant consumer of LUTs and registers, accounting for 20.5% of the total usage. The L2 cache also plays a significant role, utilizing 7.8% of LUTs and 6.8% of registers. The bus system, responsible for communication between different components, consumes 7.9% of LUTs and 4.3% of registers. Our proposed Thread Manager has a negligible impact of 0.3% of LUTs and 0.7% of registers.

Looking at the individual Rocket core, the core itself takes 26.3%, and the Floating-Point Unit (FPU) takes 38.2% of LUTs. This indicates the complexity of the core's logic and the floating-point operations. For registers, the FPU (30.1%) and data cache (DCache) (28.0%) are the primary users, highlighting the importance of fast on-chip memory for storing data and intermediate results. The hybrid cache controller only increases 0.9% of the LUTs and 2.1% of registers.

C. PERFORMANCE ANALYSIS

Table 2 compares matrix multiplication algorithms, showcasing the performance of iterative and Divide and Conquer methods in single and multi-thread implementations with the support of our proposed system and adapted software. The results reveal that both algorithms experience a significant

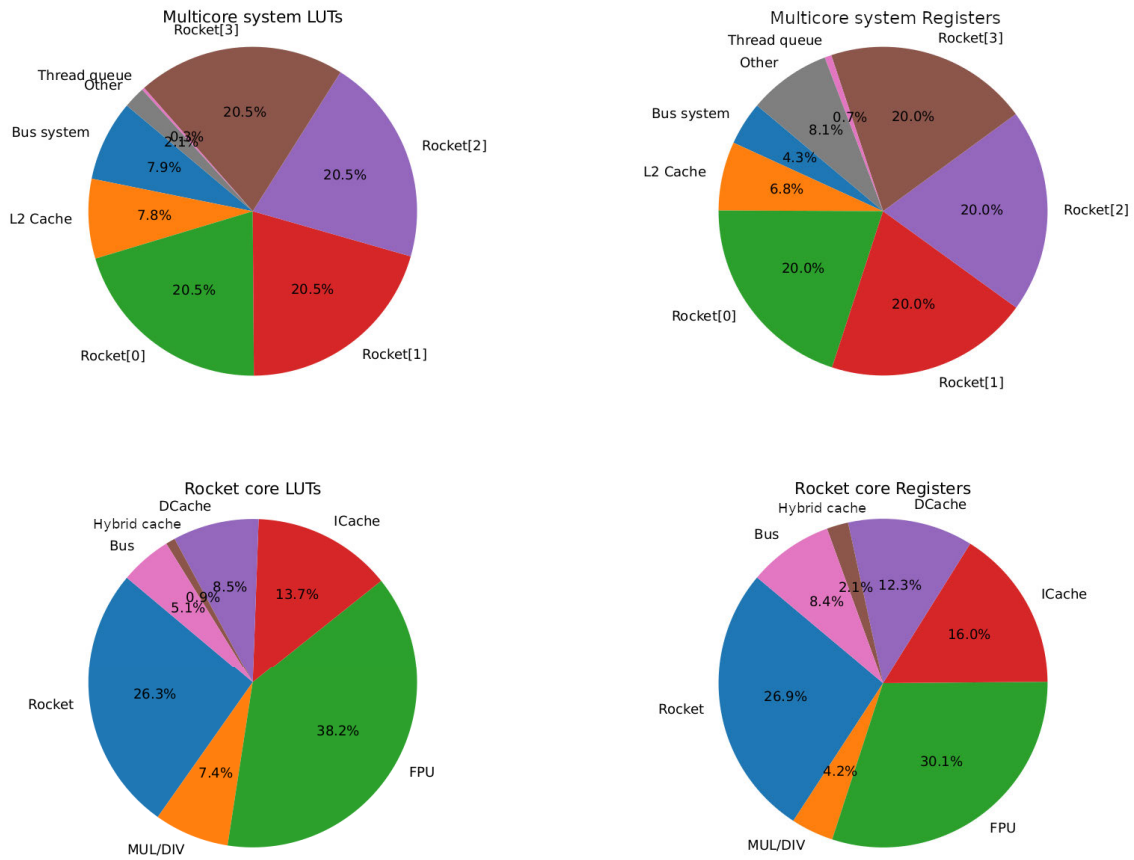


FIGURE 11. Resource consumption of for core system.

reduction in computation time when executed using multiple threads. This highlights the substantial advantage of parallel processing for matrix multiplication, a computationally intensive operation. Interestingly, our suggested approach, the Divide and Conquer algorithm, surpasses the iterative method in any case. This indicates that despite the Divide and Conquer algorithm exhibiting higher theoretical complexity, it performs better by taking advantage of the hybrid cache.

Table 3 illustrates the performance differences between single-thread and multi-thread implementations of 2D convolution across various input sizes. Multi-threading provides a substantial performance boost, with considerably faster computation times than the single-thread approach across all input sizes. This is due to the effective use of parallel processing and the support of a hybrid cache, where multiple threads work concurrently to complete the task with less bottleneck.

Additionally, the multi-thread version exhibits better instruction-level parallelism, meaning it can execute more instructions simultaneously. This is reflected in the lower Cycles Per Instruction (CPI) values compared to the single-thread version. The speedup achieved by multi-threading, indicating how much faster it is than single-threading, is consistently above 3 $\times$, demonstrating significant performance gains.

Table 4 illustrates the performance of the CRC32 checksum algorithm in both single-thread with single CRC-32 kernel (1-kernel) and single/multi-thread when performing four CRC-32 kernels (4-kernel) configurations across various text lengths. It's clear that while the multi-thread version processes more instructions due to the workload being split across multiple cores, it significantly outperforms the single-thread version in execution time.

The multi-thread implementation consistently completes the CRC32 calculation much faster, with a speedup factor ranging from 4.20 to 4.60 across different text lengths. This substantial improvement showcases the power of parallelism in accelerating computationally intensive tasks, especially with our proposed system. The significant increase in throughput is achieved through parallel processing with the help of the hybrid cache, which reduces bottlenecks on the shared bus and L2 cache.

The authors in [29], [33], [34], and [35] proposed different designs of multithread-supported RISC-V architecture. Their design relies on their specific RISC-V architecture, which leads to different resource consumption and performance. In addition, our approach can be applied to different kinds of RISC-V cores. In this work, to provide a fair comparison, we only analyze the relation between the hardware overhead and the speed up between four cores and single core

TABLE 2. Matrix multiplication evaluation.

Matrix size	Instruction count	Iterative				
		Time (cycles)		Cycles / instruction		Speed up
		Single-thread	Multi-thread	Single-thread	Multi-thread	
8	6,444	32,324	7,578	5.02	1.17	4.27
16	50,271	247,612	64,662	4.92	1.28	3.83
32	496,942	2,036,081	485,057	4.09	0.98	4.19
64	3,953,166	15,225,460	4,134,107	3.85	1.05	3.68

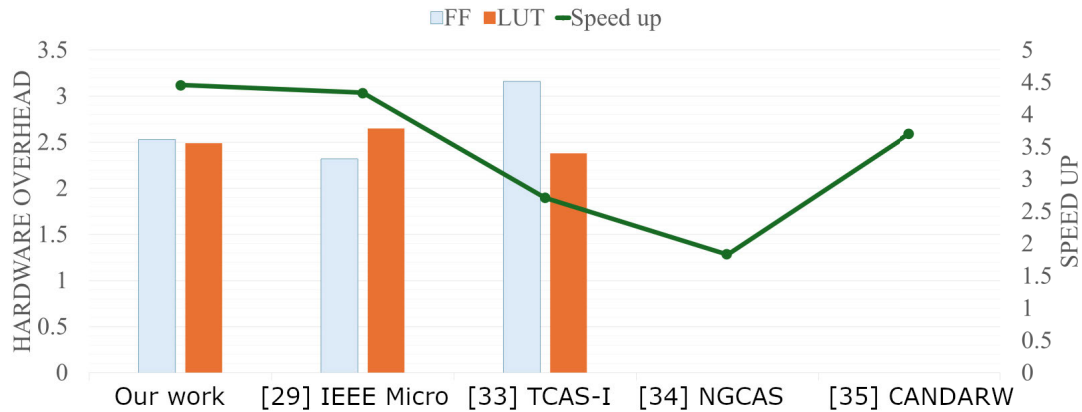
Matrix size	Instruction count	Divide and Conquer				
		Time (cycles)		Cycles / instruction		Speed up
		Single-thread	Multi-thread	Single-thread	Multi-thread	
8	6,290	40,940	7,395	6.51	1.17	5.53
16	53,271	247,612	51,186	4.65	0.96	4.83
32	406,451	2,036,081	380,012	5.01	0.93	5.35
64	3,966,614	17,820,396	3,202,560	4.49	0.81	5.56

TABLE 3. 2D Convolution evaluation.

Filter size	Instruction count	Time (cycles)		Cycles / instruction		Speed up
		Single-thread	Multi-thread	Single-thread	Multi-thread	
4	1,329,915	3,439,488	873,728	2.59	0.66	3.94
8	4,319,735	11,581,775	3,177,560	2.68	0.74	3.64
16	15,886,598	43,059,873	11,660,914	2.71	0.73	3.69
32	78,650,426	165,655,514	47,590,906	2.11	0.61	3.48

TABLE 4. CRC32 evaluation.

Text length (Byte)	Instruction count		Time (cycles)			Throughput (Bytes/cycle)			Speed up
	1-kernel	4-kernel	1-kernel	4-kernel		1-kernel	4-kernel		
				Single-thread	Multi-thread		Single-thread	Multi-thread	
256	3,080	12,314	5,381	22,085	5,164	0.05	0.01	0.05	4.28
512	6,152	24,602	11,289	44,109	9,772	0.05	0.01	0.05	4.51
1024	12,296	49,178	22,553	86,115	20,519	0.05	0.01	0.05	4.20
2048	28,680	114,714	49,177	188,494	41,005	0.04	0.01	0.05	4.60



*Assume that the resource consumption and the speed of single core is 1.

FIGURE 12. Hardware overhead and speed up of four cores over single core.

design. Figure 12 illustrates the trade-off between hardware overhead, measured by Flip-Flops (FF) and Look-Up Tables (LUT), and speed-up achieved in multicore systems compared to a single core. In this chart, the papers [34], [35] do not provide information on resource utilization. It is clear that increasing cores generally boosts speed-up; it also escalates hardware overhead. Our work offers significant speed-up with relatively low overhead. The proposal in [29]

offers nearly the same speed-up and overhead compared to our work. It emphasizes that the proposed design in [29] includes a co-processor to boost their performance. In this work, we focus on improving the RISC-V's performance in a multicore system with low resource overhead. If the user wants to improve the performance further, additional co-processors can be integrated to achieve the requirements. In addition, the designs in [29], [33], [34], and [35] include

complex hardware to manage threads inside each RISC-V core. Therefore, when quadrupling the number of cores, there are gaps between the overhead of LUT and FF. In our case, the additional hardware only occupies 0.9% of LUTs and 2.1% of FFs (as revealed in Figure 11); the hardware overheads of LUTs and FFs are nearly the same.

V. CONCLUSION

This paper presents a hardware/software co-design methodology aimed at mitigating performance bottlenecks in RISC-V-based multicore systems while maintaining minimal resource utilization. The proposed method does not target a specific RISC-V core architecture, enabling application across various RISC-V projects. Furthermore, through its integration into a complete RISC-V system, we have demonstrated our approach's practicality and real-world applicability. To validate the efficiency of the proposed methodology, we implemented a RISC-V multicore prototype platform incorporating the proposed architecture on an FPGA. We also suggest a programming scheme that can take advantage of our suggested system. We then evaluated multiple applications, including matrix multiplication, 2D convolution, and CRC-32, and provided a detailed analysis of the results. Through extensive experimental evaluation on the FPGA-based platform, we confirmed that the system's performance utilizing our proposed methodology exhibits an improvement of approximately 400% over systems without it. In addition, the implemented hybrid data cache and thread manager only increase less than 1% of the total resource of the whole system, making it suitable for a wide range of applications, from the Internet of Things (such as artificial intelligence on edge) to high-performance computing.

This work still has some limitations that open opportunities for further development. Firstly, it only focuses on enhancing RISC-V-based multicore multithreaded systems. Although it allows for integrating tightly coupled accelerators, we have yet to take any steps on it. In the future, we will shift our focus to this aspect. Secondly, the security issues on the multicore system have yet to be considered in this project. It could make the system vulnerable to attacks. Finally, our proposed thread manager allows flexibility in the thread's scheduling. Therefore, in the future, we would like to investigate adaptive cache strategies based on artificial intelligence and machine learning to improve the performance of our system further.

REFERENCES

- [1] Y.-W. Hung, Y.-T. Chang, S.-Y. Lee, C.-C. Lin, and G.-S. Shieh, "An energy-efficient and programmable RISC-V CNN coprocessor for real-time epilepsy detection and identification on wearable devices," in *Proc. IEEE Int. Conf. Consum. Electron.-Taiwan (ICCE-TW)*, Sep. 2021, pp. 1–2.
- [2] Q. Liang, S. Xie, and B. Cai, "Intelligent home IoT intrusion detection system based on RISC-V," in *Proc. IEEE 3rd Int. Conf. Power, Electron. Comput. Appl. (ICPECA)*, Jan. 2023, pp. 296–300.
- [3] J. Lee, H. Chen, J. Young, and H. Kim, "RISC-V FPGA platform toward ROS-based robotics application," in *Proc. 30th Int. Conf. Field-Program. Log. Appl. (FPL)*, Aug. 2020, p. 370.
- [4] E. Matthews and L. Shannon, "TAIGA: A new RISC-V soft-processor framework enabling high performance CPU architectural features," in *Proc. 27th Int. Conf. Field Program. Log. Appl. (FPL)*, Sep. 2017, pp. 1–4.
- [5] H. Miyazaki, T. Kanamori, M. A. Islam, and K. Kise, "RVCoreP: An optimized RISC-V soft processor of five-stage pipelining," *IEICE Trans. Inf. Syst.*, vol. E103.D, no. 12, pp. 2494–2503, Dec. 2020.
- [6] D. Rossi, F. Conti, A. Marongiu, A. Pullini, I. Loi, M. Gautschi, G. Tagliavini, A. Capotondi, P. Flatresse, and L. Benini, "PULP: A parallel ultra low power platform for next generation IoT applications," in *Proc. IEEE Hot Chips Symp. (HCS)*, Aug. 2015, pp. 1–39.
- [7] A. Amid, D. Biancolin, A. Gonzalez, D. Grubb, S. Karandikar, H. Liew, A. Magyar, H. Mao, A. Ou, N. Pemberton, P. Rigge, C. Schmidt, J. Wright, J. Zhao, Y. S. Shao, K. Asanovic, and B. Nikolic, "Chipyard: Integrated design, simulation, and implementation framework for custom SoCs," *IEEE Micro*, vol. 40, no. 4, pp. 10–21, Jul. 2020.
- [8] F. Conti, G. Paulin, A. Garofalo, D. Rossi, A. Di Mauro, G. Rutishauser, G. Ottavi, M. Eggiman, H. Okuhara, and L. Benini, "Marsellus: A heterogeneous RISC-V AI-IoT end-node SoC with 2–8 B DNN acceleration and 30%-boost adaptive body biasing," *IEEE J. Solid-State Circuits*, vol. 59, no. 1, pp. 128–142, Jan. 2024.
- [9] F. Samie, L. Bauer, and J. Henkel, "From cloud down to things: An overview of machine learning in Internet of Things," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 4921–4934, Jun. 2019.
- [10] F. Wu, J. Tian, G. Yang, H. Gui, and Y. Luo, "Design of RISC-V heterogeneous multi-core SOC architecture for edge computing for power applications," in *Proc. IEEE 6th Inf. Technol., Netw., Electron. Autom. Control Conf. (ITNEC)*, vol. 6, Mar. 2023, pp. 701–708.
- [11] H. U. Suluhan, S. Gener, A. Fusco, J. Mack, I. Dagli, M. Belviranli, C. Edemen, and A. Akoglu, "A runtime manager integrated emulation environment for heterogeneous SoC design with RISC-V cores," in *Proc. IEEE Int. Parallel Distrib. Process. Symp. Workshops (IPDPSW)*, May 2024, pp. 23–30.
- [12] M. Wang, T. Ta, L. Cheng, and C. Batten, "Efficiently supporting dynamic task parallelism on heterogeneous cache-coherent systems," in *Proc. ACM/IEEE 47th Annu. Int. Symp. Comput. Archit. (ISCA)*, May 2020, pp. 173–186.
- [13] M. Makni, M. Baklouti, S. Niar, M. W. Jmal, and M. Abid, "A comparison and performance evaluation of FPGA soft-cores for embedded multi-core systems," in *Proc. 11th Int. Design Test Symp. (IDT)*, Dec. 2016, pp. 154–159.
- [14] S. Riedel, M. Cavalcante, R. Andri, and L. Benini, "MemPool: A scalable manycore architecture with a low-latency shared L1 memory," *IEEE Trans. Comput.*, vol. 72, no. 12, pp. 3561–3575, Dec. 2023.
- [15] B. Karunatharaka, S. Kamal, and G. S. Sharvani, "Minimal multi-tasking operating system for RISC-V," in *Proc. 4th Int. Conf. Adv. Comput., Commun. Control Netw. (ICAC3N)*, Dec. 2022, pp. 2267–2270.
- [16] A. Serrano-Cases, F. Restrepo-Calle, S. Cuenca-Asensi, and A. Martínez-Álvarez, "Softerror mitigation for multi-core processors based on thread replication," in *Proc. IEEE Latin Amer. Test Symp. (LATS)*, Mar. 2019, pp. 1–5.
- [17] C. Bechara, A. Berhault, N. Ventrux, S. Chevobbe, Y. Lhuillier, R. David, and D. Etiemble, "A small footprint interleaved multithreaded processor for embedded systems," in *Proc. 18th IEEE Int. Conf. Electron., Circuits, Syst.*, Dec. 2011, pp. 685–690.
- [18] F. Conti, D. Rossi, G. Paulin, A. Garofalo, A. Di Mauro, G. Rutishauser, G. M. Ottavi, M. Eggimann, H. Okuhara, V. Huard, O. Montfort, L. Jure, N. Exibard, P. Gouedo, M. Louvat, E. Botte, and L. Benini, "A 12.4 TOPS/W @ 136 GOPS AI-IoT system-on-chip with 16 RISC-V, 2-to-8b precision-scalable DNN acceleration and 30%-boost adaptive body biasing," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 21–23.
- [19] C. Jie, I. Loi, L. Benini, and D. Rossi, "Energy-efficient two-level instruction cache design for an ultra-low-power multi-core cluster," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2020, pp. 1734–1739.
- [20] V. Jain, S. Giraldo, J. D. Roose, L. Mei, B. Boons, and M. Verhelst, "TinyVers: A tiny versatile system-on-chip with state-retentive eMRAM for ML inference at the extreme edge," *IEEE J. Solid-State Circuits*, vol. 58, no. 8, pp. 2360–2371, Aug. 2023.

- [21] D. Rossi, F. Conti, M. Eggiman, A. D. Mauro, G. Tagliavini, S. Mach, M. Guermandi, A. Pullini, I. Loi, J. Chen, E. Flamand, and L. Benini, "Vega: A ten-core SoC for IoT endnodes with DNN acceleration and cognitive wake-up from MRAM-based state-retentive sleep mode," *IEEE J. Solid-State Circuits*, vol. 57, no. 1, pp. 127–139, Jan. 2022.
- [22] S. Han, Q. Wang, and Y. Jiang, "MRAM-based cache system design and policy optimization for RISC-V multi-core CPUs," *IEEE Trans. Magn.*, vol. 59, no. 6, pp. 1–14, Jun. 2023.
- [23] G. K. Chen, P. C. Knag, C. Tokunaga, and R. K. Krishnamurthy, "An eight-core RISC-V processor with compute near last level cache in Intel 4 CMOS," *IEEE J. Solid-State Circuits*, vol. 58, no. 4, pp. 1117–1128, Apr. 2023.
- [24] A. Kamaleldin, S. Hesham, and D. Göhringer, "Towards a modular RISC-V based many-core architecture for FPGA accelerators," *IEEE Access*, vol. 8, pp. 148812–148826, 2020.
- [25] V. Potocnik, L. Colagrande, T. Fischer, L. Bertaccini, D. J. Pagliari, A. Burrello, and L. Benini, "Optimizing foundation model inference on a many-tiny-core open-source RISC-V platform," *IEEE Trans. Circuits Syst. Artif. Intell.*, vol. 1, no. 1, pp. 37–52, Sep. 2024.
- [26] A. Kamaleldin and D. Göhringer, "AGILER: An adaptive heterogeneous tile-based many-core architecture for RISC-V processors," *IEEE Access*, vol. 10, pp. 43895–43913, 2022.
- [27] X. Dong, B. De Salvo, M. Li, C. Liu, Z. Qu, H. T. Kung, and Z. Li, "SplitNets: Designing neural architectures for efficient distributed computing on head-mounted systems," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 12549–12559.
- [28] L. Du, Y. Du, Y. Li, J. Su, Y.-C. Kuan, C.-C. Liu, and M. F. Chang, "A reconfigurable streaming deep convolutional neural network accelerator for Internet of Things," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 65, no. 1, pp. 198–208, Jan. 2018.
- [29] A. Cheikh, S. Sordillo, A. Mastrandrea, F. Menichelli, G. Scotti, and M. Olivieri, "Klessydra-T: Designing vector coprocessors for multi-threaded edge-computing cores," *IEEE Micro*, vol. 41, no. 2, pp. 64–71, Mar. 2021.
- [30] H. Mo, W. Zhu, W. Hu, Q. Li, A. Li, S. Yin, S. Wei, and L. Liu, "A 12.1 TOPS/W quantized network acceleration processor with effective-weight-based convolution and error-compensation-based prediction," *IEEE J. Solid-State Circuits*, vol. 57, no. 5, pp. 1542–1557, May 2022.
- [31] B. Mazzoni, S. Benatti, L. Benini, and G. Tagliavini, "Efficient transform algorithms for parallel ultra-low-power IoT end nodes," *IEEE Embedded Syst. Lett.*, vol. 13, no. 4, pp. 210–213, Dec. 2021.
- [32] M. A. Islam and K. Kise, "Efficient resource shared RISC-V multicore processor," in *Proc. IEEE 14th Int. Symp. Embedded Multicore/Many-Core Systems-Chip (MCSoc)*, Dec. 2021, pp. 366–372.
- [33] Y. Eni, S. Greenberg, and Y. Ben-Shimol, "Efficient hint-based event (EHE) issue scheduling for hardware multithreaded RISC-V pipeline," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 69, no. 2, pp. 735–745, Feb. 2022.
- [34] M. Olivieri, A. Cheikh, G. Cerutti, A. Mastrandrea, and F. Menichelli, "Investigation on the optimal pipeline organization in RISC-V multi-threaded soft processor cores," in *Proc. New Gener. CAS (NGCAS)*, Sep. 2017, pp. 45–48.
- [35] Y. Nojiri and N. Yamasaki, "A design of multithreaded RISC-V processor for real-time system," in *Proc. 11th Int. Symp. Comput. Netw. Workshops (CANDARW)*, Nov. 2023, pp. 31–37.



KHAI-DUY NGUYEN (Graduate Student Member, IEEE) received the B.Sc. degree in electronics from the University of Danang-University of Science and Technology, Da Nang, Vietnam, in 2020, and the M.S. degree in electronics from The University of Electro-Communications (UEC), Tokyo, Japan, in 2024, where he is currently pursuing the Ph.D. degree in information and network engineering.



NGUYEN THE BINH (Graduate Student Member, IEEE) received the B.Eng. degree in computer engineering from the Faculty of Computer Science and Engineering, Vietnam National University Ho Chi Minh City. He is currently a Teaching Assistant with the Faculty of Computer Science and Engineering. His research interests include digital system design of microprocessor architecture, hardware accelerator, and post-quantum cryptography.



KHAI-MINH MA (Graduate Student Member, IEEE) received the B.Sc. degree in electronics and telecommunication engineering, in 2018. He is currently pursuing the Master of Electronics Engineering degree in micro-electronics and IC design with the University of Science, Vietnam National University Ho Chi Minh City. He is also a Researcher with the DESLab and a Teaching Assistant with the Electronics Department, Faculty of Electronics and Telecommunications, University of Science, Vietnam National University Ho Chi Minh City. His research interests include designing digital systems on FPGA and integrated circuits.



TRI-DUC TA (Graduate Student Member, IEEE) received the B.Eng. degree in electronics and telecommunications engineering from Ton Duc Thang University, the B.A. degree in English linguistics and literature from the University of Social Sciences and Humanities, Vietnam National University Ho Chi Minh City, the M.Sc. degree in electronics engineering from the University of Sciences, Vietnam National University Ho Chi Minh City, and the M.A. degree in English linguistics from the Graduate Academy of Social Sciences, Vietnam Academy of Social Sciences. He is currently a Lecturer with the Faculty of Computer Engineering, University of Information Technology, Vietnam National University Ho Chi Minh City. His research interests include the design of digital systems on FPGA and integrated circuits and digital signal processing.



BINH KIEU-DO-NGUYEN (Graduate Student Member, IEEE) received the B.Sc. and M.S. degrees from the Department of Computer Science and Engineering, Ho Chi Minh City University of Technology (HCMUT), Ho Chi Minh City, Vietnam, in 2017 and 2019, respectively. He is currently pursuing the Ph.D. degree in information and network engineering with The University of Electro-Communications (UEC), Tokyo, Japan. From 2017 to 2021, he was a Lecturer Assistant with HCMUT. Since 2022, he has been a Research Assistant with UEC. His research interests include computer architecture, hardware security, and digital systems design.



University of Science, Vietnam National University Ho Chi Minh City. His research interests include IC design, SoC design, digital signal processing, neuromorphic computing, and biomedical electronics.

DUK-HUNG LE (Member, IEEE) received the B.Sc. degree in physics and the M.Sc. degree in electronic physics from the University of Science, Vietnam National University Ho Chi Minh City, in 2001 and 2005, respectively, and the Ph.D. degree in advanced science and engineering from The University of Electro-Communications (UEC), Tokyo, Japan, in 2013. He is currently the Head of the Electronics Department and the DESLAB, Faculty of Electronics and Telecommunications,



system-onchip by integrating various information processing hardware, and develops a high-performance computational circuit realized with a small number of elements. His research interests include hardware system design implementation by FPGA and integrated circuits.

CONG-KHA PHAM (Senior Member, IEEE) received the B.S., M.S., and Ph.D. degrees in electronics engineering from Sophia University, Tokyo, Japan. He is currently a Professor with the Department of Information and Network Engineering, The University of Electro-Communications (UEC), Tokyo. Integrated Circuit Design Laboratory (Pham Lab), UEC, educates the design, implementation, and evaluation of hardware systems and VLSI, aims to design a



he was a Research Assistant with the Cyber-Physical Security Research Center (CPSEC), National Institute of Advanced Industrial Science and Technology (AIST), Tokyo. Since April 2022, he has been an Assistant Professor with the Department of Computer and Network Engineering, UEC. His research interests include digital signal processing, computer architecture, cyber-security, and ultra-low power systems-on-a-chip.

TRONG-THUC HOANG (Member, IEEE) received the B.Sc. and M.S. degrees in electronic engineering from Ho Chi Minh City University of Science (HCMUS), Ho Chi Minh City, Vietnam, in 2012 and 2017, respectively, and the Ph.D. degree in engineering from The University of Electro-Communications (UEC), Tokyo, Japan, in 2022. From 2012 to 2017, he was a Lecturer Assistant with HCMUS. From 2019 to 2020, he was a Research Assistant with UEC. From 2019 to 2022,

...